

# The Prompt Ladder

Ahmed Elgharably — FlyRank ML Internship — General AI Fluency, Week 2

---

## Context

I picked a prompt from my ML track work: asking an AI assistant to help me analyze the FlyRank starter dataset for content refresh opportunities. This is something I actually need to do for my capstone.

---

## Baseline (Version 0): The Weak Prompt

### Prompt:

Analyze this dataset for me.

### Output (representative excerpt):

I'd be happy to help you analyze your dataset! To get started, I'll need a few details:

1. What format is the data in? (CSV, Excel, JSON, etc.)
2. What is the dataset about? (Sales, customer behavior, website analytics, etc.)
3. What kind of analysis are you looking for? (Summary statistics, trends, correlations, predictions)
4. Are there any specific columns or variables you're most interested in?

Please share the dataset or describe its structure, and I'll guide you through the analysis step by step!

---

## Version 1: Add a Clearer Goal

**What changed in the prompt:** Added a specific analytical goal instead of the vague “analyze this dataset.”

### Prompt:

Help me identify which content pages in this dataset are the best candidates for a refresh or update.

### Output (representative excerpt):

To identify the best content refresh candidates, I'll need to know more about your dataset. Generally, good refresh candidates are pages that:

- Have high traffic but outdated content
- Show declining performance over time
- Rank well but have low engagement

Can you share the column names or a sample of the data? Specifically, I'm looking for columns related to:

- Traffic metrics (views, impressions, clicks)
- Content age or last updated date
- Performance trends (growth, decline, stable)
- Engagement metrics (bounce rate, time on page)

**What actually improved in the output:** The AI stopped asking “what do you want” and started listing relevant criteria for content refresh candidates. It now assumes I’m working with web content and mentions specific signal types (traffic, age, trends, engagement) that are actually relevant to my problem.

**What still failed:** It still doesn’t know what data I have, so it can’t give me actionable code or specific recommendations. It’s giving generic advice that applies to any content dataset, not my specific 30K-row FlyRank starter file.

**What I would try next:** Give it the actual column names and data structure so it can write specific code instead of generic guidance.

---

## Version 2: Add Real Context

**What changed in the prompt:** Added the actual dataset structure, column names, and a description of what the data contains.

### Prompt:

Help me identify which content pages in this dataset are the best candidates for a refresh or update. The dataset is `content_refresh_anonymized.csv` with 30,000 rows and these key columns:

- `content_id`, `client_id` (identifiers)
- `impressions_90d`, `clicks_90d`, `sessions_90d` (traffic)
- `avg_position`, `ctr` (search performance)
- `content_age_days`, `days_since_last_update` (freshness)
- `word_count`, `content_type` (content metadata)
- `trend_direction` (up / down / stable)
- `engagement_rate`, `scroll_rate` (user engagement)

### Output (representative excerpt):

Great! With those columns, here’s how I’d identify refresh candidates. I’ll write Python code to flag pages in three priority tiers:

**Tier 1 — Urgent:** `trend_direction == 'down' AND impressions_90d >= 100` **Tier 2 — Stale but visible:** `content_age_days >= 180 AND impressions_90d >= 500` **Tier 3 — Low CTR opportunity:** `avg_position <= 20 AND ctr < 0.5 AND impressions_90d >= 500`

```
python > import pandas as pd > df = pd.read_csv('content_refresh_anonymized.csv') >
> urgent = df[(df['trend_direction'] == 'down') & (df['impressions_90d'] >= 100)] >
> stale_visible = df[(df['content_age_days'] >= 180) & (df['impressions_90d'] >= 500)] >
> low_ctr = df[(df['avg_position'] <= 20) & (df['ctr'] < 0.5) & (df['impressions_90d'] >= 500)] >
```

This gives you three actionable segments. Want me to combine them into a single ranked score?

**What actually improved in the output:** Now the AI wrote actual Python code using my real column names and proposed specific, actionable thresholds. It created three distinct segments instead of generic advice. The code is runnable against my actual file.

**What still failed:** The thresholds (100 impressions, 180 days, position 20) feel arbitrary — the AI guessed them. It also doesn’t explain WHY these thresholds make sense for my specific business context, and it doesn’t tell me how to rank pages when a page falls into multiple tiers.

**What I would try next:** Define who will use this output and what their constraints are, so the

thresholds and ranking logic match real decision-making capacity.

---

### Version 3: Add a Defined Audience + Constraints

**What changed in the prompt:** Added who will act on this (a content reviewer with limited weekly capacity) and what decisions they make.

**Prompt:**

Help me identify which content pages in this dataset are the best candidates for a refresh or update. The dataset is `content_refresh_anonymized.csv` with 30,000 rows and these key columns: [same as V2]

The output is for a content team that can only review 20–50 pages per week. They need a ranked queue so they know which pages to look at first. The team can take these actions: refresh content, expand thin sections, protect a declining winner, prune obsolete pages, or monitor and wait.

**Output (representative excerpt):**

Perfect — with a 20–50 page weekly capacity, you need a single ranked score, not just buckets. Here's a weighted scoring approach that produces one priority queue:

```
python > import pandas as pd > import numpy as np > > df =
pd.read_csv('content_refresh_anonymized.csv') > > # Normalize key signals to
0-1 > df['visibility_score'] = np.clip(df['impressions_90d'] / 1000, 0, 1) >
df['decline_score'] = (df['trend_direction'] == 'down').astype(int) > df['stale_score'] =
np.clip(df['content_age_days'] / 365, 0, 1) > df['ctr_gap_score'] = np.clip((0.5 - df['ctr'])
/ 0.5, 0, 1) > > # Combined refresh opportunity score > df['refresh_score'] = ( > 0.35 *
df['visibility_score'] + > 0.25 * df['decline_score'] + > 0.25 * df['stale_score'] +
> 0.15 * df['ctr_gap_score'] > ) > > # Rank and filter for minimum volume > candidates
= df[df['impressions_90d'] >= 100].sort_values('refresh_score', ascending=False) > top_50
= candidates.head(50) >
```

This gives you a single ranked list of 50 pages per week. Each page gets a score from 0–1, and the weights favor visible pages that are declining or stale.

**What actually improved in the output:** The AI moved from static buckets to a **single ranked score** — exactly what a team with limited capacity needs. It added minimum volume filters (`impressions_90d >= 100`) to avoid noise, and it weighted the factors so the ranking reflects business priority. The output now matches the real workflow (“here are your top 50 this week”).

**What still failed:** The weights (0.35, 0.25, 0.25, 0.15) are arbitrary — the AI invented them. A content team wouldn't know if 0.35 for visibility is too high or too low. Also, there's no explanation of WHY a specific page got its score — the reviewer sees a number but not the reason.

**What I would try next:** Add examples of what “good” looks like and ask for reason codes so reviewers understand why each page was flagged.

---

### Version 4: Add Examples of What Good Looks Like + Reason Codes

**What changed in the prompt:** Added examples of specific page profiles that should rank high, and asked for human-readable reason codes alongside the score.

### Prompt:

Help me identify which content pages in this dataset are the best candidates for a refresh or update. The dataset is `content_refresh_anonymized.csv` with 30,000 rows and these key columns: [same as V2]

The output is for a content team that can only review 20–50 pages per week. They need a ranked queue so they know which pages to look at first.

Here are examples of pages that **SHOULD** rank high:

- A page with 800 impressions, `trend_direction='down'`, `content_age_days=250`, `ctr=0.3`
- A page with 1200 impressions, `trend_direction='stable'`, `content_age_days=300`, `ctr=0.2`

Here are examples of pages that should **NOT** rank high:

- A page with 50 impressions, `trend_direction='down'`, `content_age_days=400` (too little volume)
- A page with 2000 impressions, `trend_direction='up'`, `content_age_days=30` (already performing well)

For each page in the top 50, include a short reason code explaining **WHY** it was flagged (e.g., “declining\_with\_demand”, “stale\_visible\_page”, “low\_ctr\_opportunity”).

### Output (representative excerpt):

With those examples, I can see the pattern: high volume + decline/stale/weak CTR = priority. Low volume or already-growing pages = deprioritize. Here's the updated code with reason codes:

```
python > def get_reason_codes(row): > codes = [] > if row['trend_direction'] == 'down' and row['impressions_90d'] >= 100: > codes.append('declining_with_demand') > if row['content_age_days'] >= 180 and row['impressions_90d'] >= 500: > codes.append('stale_visible_page') > if row['avg_position'] <= 20 and row['ctr'] < 0.5 and row['impressions_90d'] >= 500: > codes.append('low_ctr_opportunity') > if row['word_count'] < 1200 and row['impressions_90d'] >= 250: > codes.append('thin_visible_page') > return ', '.join(codes) if codes else 'monitor' > > df['reason_codes'] = df.apply(get_reason_codes, axis=1) > > # Score: only pages with at least one reason code get ranked > df['refresh_score'] = df['reason_codes'].apply(lambda x: len(x.split(', ')) if x != 'monitor' else 0) > > # Weight by volume > df['refresh_score'] = df['refresh_score'] * np.log1p(df['impressions_90d']) > > candidates = df[df['reason_codes'] != 'monitor'].sort_values('refresh_score', ascending=False) > top_50 = candidates.head(50)[['content_id', 'impressions_90d', 'trend_direction', 'content_age_days', 'ctr', 'refresh_score', 'reason_codes']] >
```

**Sample output:**

| content_id | impressions_90d | trend  | age_days | ctr  | score | reason_codes                              |
|------------|-----------------|--------|----------|------|-------|-------------------------------------------|
| abc123     | 850             | down   | 245      | 0.28 | 4.2   | declining_with_demand, stale_visible_page |
| def456     | 1200            | stable | 310      | 0.18 | 3.8   | stale_visible_page, low_ctr_opportunity   |

**What actually improved in the output:** The AI now generates **human-readable reason codes** that tell the reviewer **WHY** a page was flagged. The examples I gave helped it calibrate the thresholds better — it now uses `impressions_90d >= 500` for stale pages (matching my “visible” example) and excludes low-volume pages. The output is now actionable for a human reviewer, not just a number.

**What still failed:** The scoring logic is still hand-tuned rules. It doesn't learn from historical patterns — it just counts how many reason codes apply and weights by log impressions. A real ML model could probably beat this by learning which combinations of signals actually predict decline, rather than using

fixed thresholds.

**What I would try next:** Ask the AI to frame this as a machine learning problem with proper validation, not just hand-rules.

---

## Version 5: Add Quality Criteria + Verification Requirements

**What changed in the prompt:** Added that this needs to be a proper ML approach with validation, not just rules, and asked for a leakage audit and baseline comparison.

### Prompt:

Help me identify which content pages in this dataset are the best candidates for a refresh or update. The dataset is `content_refresh_anonymized.csv` with 30,000 rows and these key columns: [same as V2]

The output is for a content team that can only review 20–50 pages per week. They need a ranked queue so they know which pages to look at first.

Examples of high-priority and low-priority pages: [same as V4]

This should be a **machine learning model**, not just hand-tuned rules. I need:

- A train/test split with client-level holdout (test on clients the model hasn't seen)
- A leakage audit: confirm no feature uses future information or the target itself
- A baseline comparison: compare the model against a transparent rule-based score
- Evaluation metrics: Precision@50, Average Precision, and ROC AUC
- The final output should still include reason codes for the top-ranked pages

### Output (representative excerpt):

Excellent — let's build this properly. Here's a full ML pipeline with client-holdout validation, leakage checks, and baseline comparison:

```
python > import pandas as pd > import numpy as np > from sklearn.model_selection
import GroupShuffleSplit > from sklearn.ensemble import RandomForestClassifier
> from sklearn.linear_model import LogisticRegression > from sklearn.metrics
import roc_auc_score, average_precision_score, precision_score > > df =
pd.read_csv('content_refresh_anonymized.csv') > > # Define target: pages that are declining
> df['is_declining'] = (df['trend_direction'] == 'down').astype(int) > > # Features (all
known BEFORE prediction time) > features = ['impressions_90d', 'clicks_90d', 'sessions_90d',
'avg_position', > > 'ctr', 'content_age_days', 'days_since_last_update',
'word_count', > > 'engagement_rate', 'scroll_rate'] > > # LEAKAGE AUDIT >
assert 'is_declining' not in features # Target not in features > assert 'trend_pct'
not in features or df['trend_pct'].isna().all() # No future trend > print("Leakage
check passed: no target or future info in features") > > # Client-level holdout
split > splitter = GroupShuffleSplit(n_splits=1, test_size=0.3, random_state=42) >
train_idx, test_idx = next(splitter.split(df, groups=df['client_id'])) > > X_train,
X_test = df.iloc[train_idx][features], df.iloc[test_idx][features] > y_train, y_test =
df.iloc[train_idx]['is_declining'], df.iloc[test_idx]['is_declining'] > > # BASELINE:
transparent rule score > baseline_score = ( > > 0.40 * np.clip(df['impressions_90d']
/ 1000, 0, 1) + > > 0.30 * (df['content_age_days'] >= 180).astype(float) + >
0.25 * (df['avg_position'] <= 10).astype(float) * (df['ctr'] < 0.5).astype(float) +
```

```

> 0.05 * (df['word_count'] < 1200).astype(float) > ) > > # MODELS > models = {
> 'logistic_regression': LogisticRegression(max_iter=1000), > 'random_forest':
RandomForestClassifier(n_estimators=100, random_state=42) > } > > results = {} >
for name, model in models.items(): > model.fit(X_train, y_train) > probs =
model.predict_proba(X_test)[: , 1] > > # Precision@50 > top_50_idx =
np.argsort(probs)[-50:] > precision_at_50 = y_test.iloc[top_50_idx].mean() > >
results[name] = { > 'roc_auc': roc_auc_score(y_test, probs), > 'avg_precision':
average_precision_score(y_test, probs), > 'precision_at_50': precision_at_50
> } > > # Baseline metrics > baseline_probs = baseline_score.iloc[test_idx]
> results['baseline'] = { > 'roc_auc': roc_auc_score(y_test, baseline_probs), >
'avg_precision': average_precision_score(y_test, baseline_probs), > 'precision_at_50':
y_test.iloc[np.argsort(baseline_probs)[-50:]].mean() > } > > print(pd.DataFrame(results).T)
> > # Reason codes for top 50 from best model > best_model = models['random_forest'] > test_df
= df.iloc[test_idx].copy() > test_df['model_prob'] = best_model.predict_proba(X_test)[: ,
1] > top_50 = test_df.nlargest(50, 'model_prob') > > def get_reason_codes(row): >
codes = [] > if row['trend_direction'] == 'down' and row['impressions_90d'] >= 100:
> codes.append('declining_with_demand') > if row['content_age_days'] >=
180 and row['impressions_90d'] >= 500: > codes.append('stale_visible_page')
> if row['avg_position'] <= 20 and row['ctr'] < 0.5 and row['impressions_90d'] >= 500:
> codes.append('low_ctr_opportunity') > return ' '.join(codes) if codes else
'model_flagged' > > top_50['reason_codes'] = top_50.apply(get_reason_codes, axis=1) >
Expected results (from starter pipeline): | Method | ROC AUC | Avg Precision | Precision@50
| |-----|-----|-----|-----| | baseline | 0.627 | 0.468 | 0.240 | | logistic regression | 0.700
| 0.522 | 0.400 | | random forest | 0.750 | 0.618 | 0.740 |

```

**What actually improved in the output:** The AI produced a **complete ML pipeline** with proper validation design: client-holdout split, leakage audit, baseline comparison, and the right metrics (Precision@50). It didn't just give code — it explained **WHY** each piece matters. The output now matches the rigor expected in the ML track capstone.

**What still failed:** The target (`is_declining = trend_direction == 'down'`) is a proxy label, not a future outcome. The assignment guide explicitly warns that this is a “beginner proxy label” and a stronger capstone should use future-window labels. The AI didn't question this — it just implemented what I asked.

**What I would try next:** Push back on the target definition and ask for a future-window label (e.g., predict decline in the NEXT 30 days using features from the PRIOR 90 days). But that requires the full warehouse dataset, not just the starter CSV.

## Final Reusable Prompt

**Role:** You are an ML engineer building a content refresh prioritization system for a digital publisher.

**Goal:** Build a ranked queue of content pages that a human reviewer should examine first, given they can only review 20–50 pages per week.

**Dataset:** `content_refresh_anonymized.csv` (30K rows) with columns: `content_id`, `client_id`, `impressions_90d`, `clicks_90d`, `sessions_90d`, `avg_position`, `ctr`, `content_age_days`, `days_since_last_update`, `word_count`, `content_type`, `trend_direction`, `engagement_rate`,

scroll\_rate.

**Actions the team can take:** refresh content, expand thin sections, protect a declining winner, prune obsolete pages, or monitor and wait.

**Requirements:**

1. Build a machine learning model, not just hand-tuned rules.
2. Use client-level holdout validation (test on unseen clients).
3. Audit for leakage: no feature can use future information or the target.
4. Compare against a transparent baseline score.
5. Report Precision@50, Average Precision, and ROC AUC.
6. For the top 50 ranked pages, include human-readable reason codes.
7. Use careful language: this is decision-support, not causal proof.

## Summary Table

| Version | Layer Added             | Output Improvement                       | What Still Failed                | Next Step            |
|---------|-------------------------|------------------------------------------|----------------------------------|----------------------|
| 0       | —                       | Asked clarifying questions               | Everything                       | Add goal             |
| 1       | Clearer goal            | Listed relevant criteria                 | Generic, no code                 | Add context          |
| 2       | Real context            | Wrote runnable Python code               | Arbitrary thresholds             | Add audience         |
| 3       | Audience + constraints  | Single ranked score for limited capacity | Arbitrary weights                | Add examples         |
| 4       | Examples + reason codes | Human-readable explanations              | Still hand-tuned rules           | Add ML rigor         |
| 5       | Quality + verification  | Full ML pipeline with validation         | Proxy label (not future outcome) | Future-window labels |

## Honest Moment

Version 3 (adding audience/constraints) actually made the output **worse** at first. The AI overcomplicated the scoring with four normalized components and invented weights (0.35, 0.25, 0.25, 0.15) that had no basis in the data. I had to explicitly ask for reason codes in Version 4 to bring the output back to something a human could trust. The lesson: more context doesn't always help if the AI fills gaps with confident guesses.

*Submitted to the General AI Fluency track thread.*